Programming Fundamentals

Module A - Introduction

Le The Anh
anhlt161@fe.edu.vn



FPT UNIVERSITY

Objectives

- Define some concepts related to programming
- Explain how to make a good software
- Understand steps to develop a software
- Explain ways for representing data
- Answer why C is the first language selected
- Understand how a C program can be translated and execute
- Discuss about notable features of the C language
- Understand a C program structure

Contents

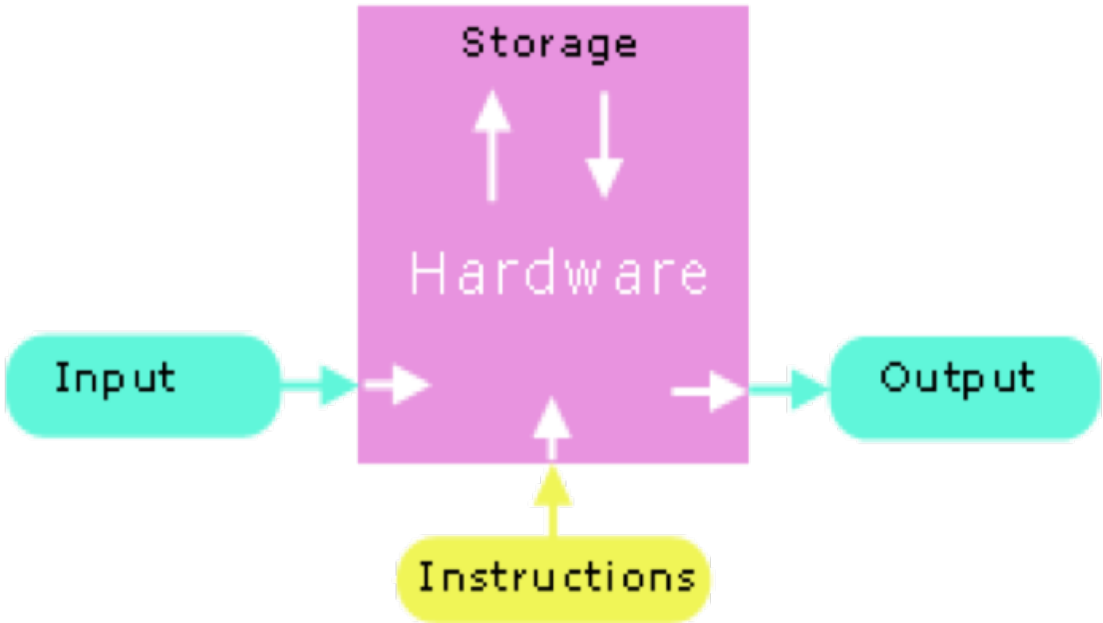
1. Basic concepts
2. How to make a good software
3. Software development steps
4. Computer hardware
5. Data units
6. Addressing information
7. Program instructions
8. Programming languages
9. Translating and executing a program
10. Why C as a first language
11. Some notable features
12. Structure of a simple C program

Basic concepts

- Information: Knowledge about something
- Data: Values are used to describe information. So, information can be called as the mean of data.
- Problem: A situation in which something is hidden
- Solve a problem: explore the hidden information
- Algorithm: A finite sequence of well-defined instructions, typically used to solve a class of specific problems or to perform a computation
- Program: A sequence of steps to find out the solution of a problem. A program is an implementation of an algorithm.

Basic concepts

- Computer program: A program executed using a computer; a set of instructions that computer hardware will execute.
- The instructions transform raw data (the input) into information and eventually into comprehensible results (the output).
- The hardware stores the data internally during this process.



- Computer software: A set of related programs.

How to make a good software?

Issues for a program/software:

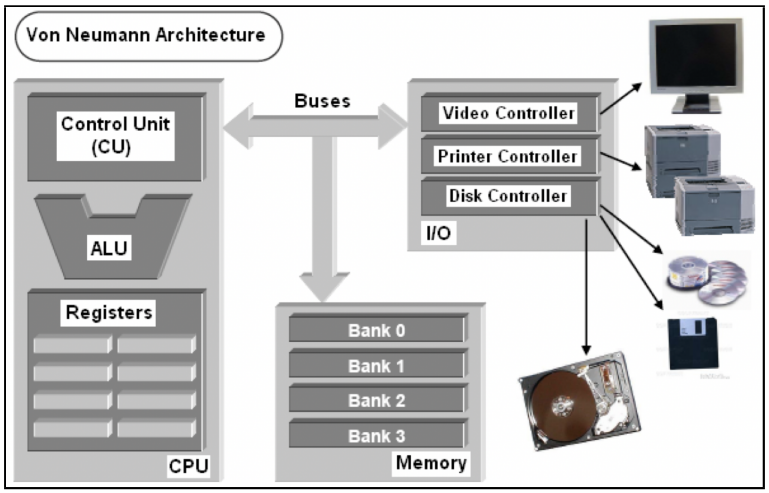
- Usability: Users can use the program to solve the problem
  - Robust and user-friendly interfaces
- Correctness: Solution must be correct
  - Comprehensive testing
- Maintainability: The program can be modified easily
  - Understandability
    - Structured programming
    - Internal documentation
- Modifiability
  - Standards compliance
- Portability: The program can run in different platforms
  - Standards compliance $\rightarrow$ Needed modifications are minimum (platform: CPU + operating system running on it)

Software Development Steps

1. Requirements: Understand the problem
2. Analysis: Identify data and tasks
3. Design: Organize folder, files, ..
4. Coding: Implementation
5. Testing: Check whether requirements are satisfied or not
6. Deploying: Install the program to user computers
7. Maintenance: Make needed modifications

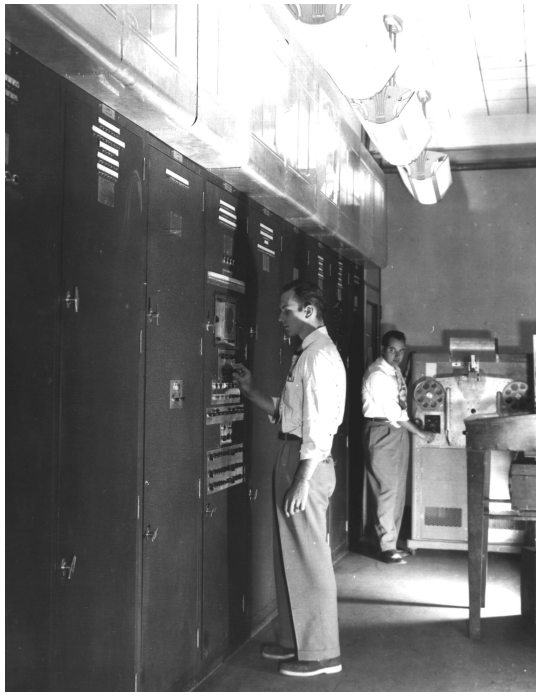
Computer hardware

- Central Processing Unit (CPU) executes program instructions serially (one at a time). A modern CPU consists of: Register, a decode unit, a control unit (CU), a arithmetic and logic unit (ALU), a floating-point accelerator (FPA).
- Primary memory: is directly accessible by the CPU, including ROM, RAM
  - ROM (Read-Only Memory): holds the instructions for starting OS. ROM is not volatile.
  - RAM (Random Access Memory): holds the program instructions and data. RAM is volatile.
- I/O devices: speaker, keyboard, mouse, monitor, ..



Data units

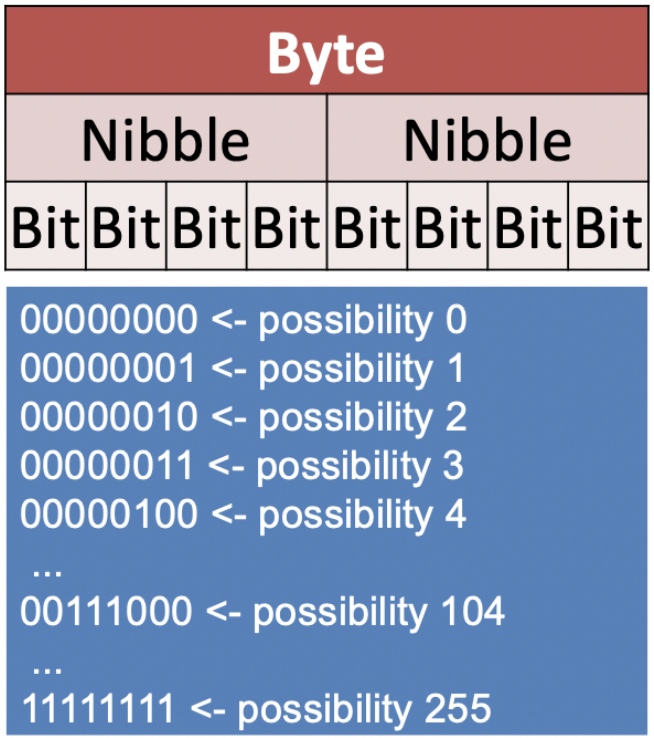
- John von Neumann selected binary (base 2) digits as the EDVAC's fundamental unit.
- The vast majority of modern computers process and store information in binary digits.
- We call a bi nary digi t a bit .



The EDVAC as installed in Building 328 at the Ballistic Research Laboratory (borrowed from Wikipedia)

Data units

- Nibble = 4 consecutive bits
- Byte = 2 nibbles = 8 bits
- Unit of memory is Byte
- The natural unit of CPU is a word whose length is number of bits of a general register within CPU (CPU memory). Word length can be 8, 16 (old CPUs), 32, 64 (current CPUs)



Addressing Information

- To access data in primary memory, program instructions refer to the addresses where that data is stored.
- Each byte of primary memory has a unique address. Addressing starts at zero, is sequential and ends at the size of primary memory less 1.
  - Exabyte (E), Petabyte (P), Terabyte (T), Gigabyte (G), Megabyte (M), Kilobyte (K), Byte (B), Bit (b)
  - 1E $= 2^{10} = 1024$P, 1P = 1024T, .., 1k = 1024 bytes, 1 byte = 8 bits.

Size:	1 Byte		1 Byte		1 Byte		...	1 Byte	
Hex:	1 Nibble	1 Nibble	1 Nibble	1 Nibble	1 Nibble	1 Nibble	...	1 Nibble	1 Nibble
Contents:							...		
Binary:	00000000_2		00000001_2		00000010_2		...	$1...111111111_2$	
Hex:	0x0000000		0x0000001		0x0000002		...	0xFFFFFFF	

- The maximum size of addressable primary memory depends upon the size of the address registers (Pentium machines where the address registers hold 32 bits the maximum size of addressable memory is about 4GB, ranging from 0 to $2^{32} - 1$)

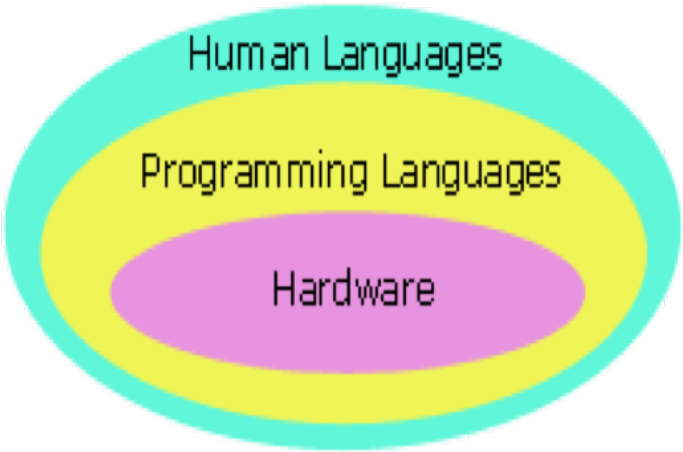
Program Instructions

- Each program instruction consists of an operation and operands.
- The CPU performs the operation on the values stored as operands or on the values stored in the operand addresses.
- Operands: constants, registers, primary memory addresses.

01001011	100110110110	011011010111
Opcode	Operand 1	Operand 2

Programming Languages

- Programs that perform relatively simple tasks and are written in assembly language contain a large number of statements.
- The higher the level, the closer to the human languages and the further from native machine languages

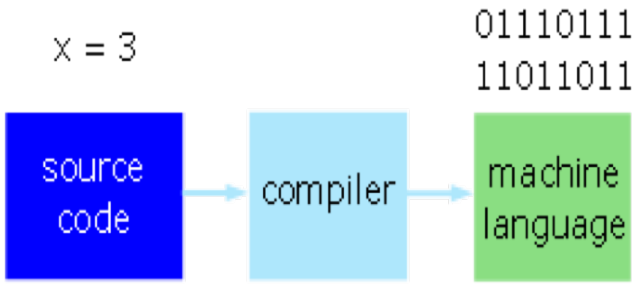


Programming Languages

- Machine languages (1940s): Native machine language. Each manufacturer has its own unique set of instructions.
- Assembly languages (late 1940s): These are readable versions of the native machine languages. The manufacturer supplies an assembly language for its machines.
- Third-generation languages. These are languages with instructions that describe how a result is to be obtained (Eg. FORTRAN 1950s; Pascal, C - 1970s)
- Fourth-generation languages. These are languages with instructions that describe what is to be done without specifying how it is to be done (Eg. SQL, Matlab)
- Fifth-generation languages are the closest to human languages. They are used for artificial intelligence, fuzzy sets, and neural networks (Eg. Mercury, ICAD).

Translating and executing a program

- When we code a program in a high level language, we write source code that can't be run by computers. The source code must be translated to binary code (machine code). This is done by using either an interpreter, or a compiler.
- Interpreters: Translate and execute one-by-one statement.
- Compilers: Translate the entire set of high level statements into an equivalent set of machine language statements (without executing any of the statements) and produce a separate executable file. We execute that file separately afterwards. Compilers can produce optimized results.



Compiling

Why C as a First Language

- C is one of the most popular languages in use globally (TIOBE index)
- Some reasons for learning programming using the C language include:
  - C is English-like,
  - C is quite compact - has a small number of keywords,
  - A large number of C programs need to be maintained,
  - C is the lowest in level of the high-level languages,
  - C is faster and more powerful than other high-level languages, and
  - The UNIX, Linux and Windows are written in C and C++.
- Comparative times for Sieve of Eratosthenes test

Language	Time to Run
Assembly	0.18 seconds
C	2.7 seconds
Basic	10 seconds

Some Notable Features

- Comments (/* */, //): We use comments to document our programs and to enhance their readability. C compilers ignore all comments.
- Whitespace: We use whitespace to improve program readability and to display the structure of our program's logic. C compilers ignore all whitespace.
- Case Sensitivity: C language is case sensitive ('A' and 'a' are totally different).

Simple C Program

```c
/*
My first program
hello.c
Le The Anh
Aug 14 2021
*/

#include <stdio.h>

int main()
{
    printf("Hello World! I'm a C program.");
    return 0;
}
```

Summary

1. Basic concepts
2. How to make a good software
3. Software development steps
4. Computer hardware
5. Data units
6. Addressing information
7. Program instructions
8. Programming languages
9. Translating and executing a program
10. Why C as a first language
11. Some notable features
12. Structure of a simple C program